

Sport Activity Recognition

YOLOv11n · Sony IMX500 AI Camera · Raspberry Pi 5

4 Sport Classes

916 Images

95.7% mAP50

25 FPS On-Device

01

Motivation & Overview

Why edge AI for sport detection?

Project Overview

Real-time sport activity detection on a Sony IMX500 AI camera

The Problem

- Manual sport coaching is time-consuming
- Cameras need cloud connectivity for AI
- Latency is too high for real-time feedback

Our Solution

- Train a compact YOLOv11n detector
- Export to Sony IMX500 edge AI chip
- Run inference directly in the camera

Key Benefit

- No cloud — fully private
- <3 ms per frame on-chip
- Runs on Raspberry Pi 5



02

Dataset

Sports AISD — Custom annotated dataset

Dataset — Sports AISD

916 images • YOLO format • 4 sport classes • 80/10/10 split

916

Total Images

4

Classes

YOLO

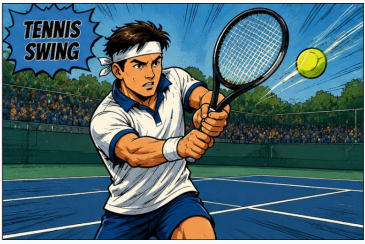
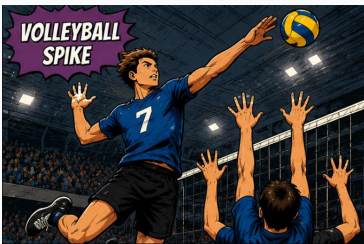
Label Format

80/10/10

Train/Val/Test

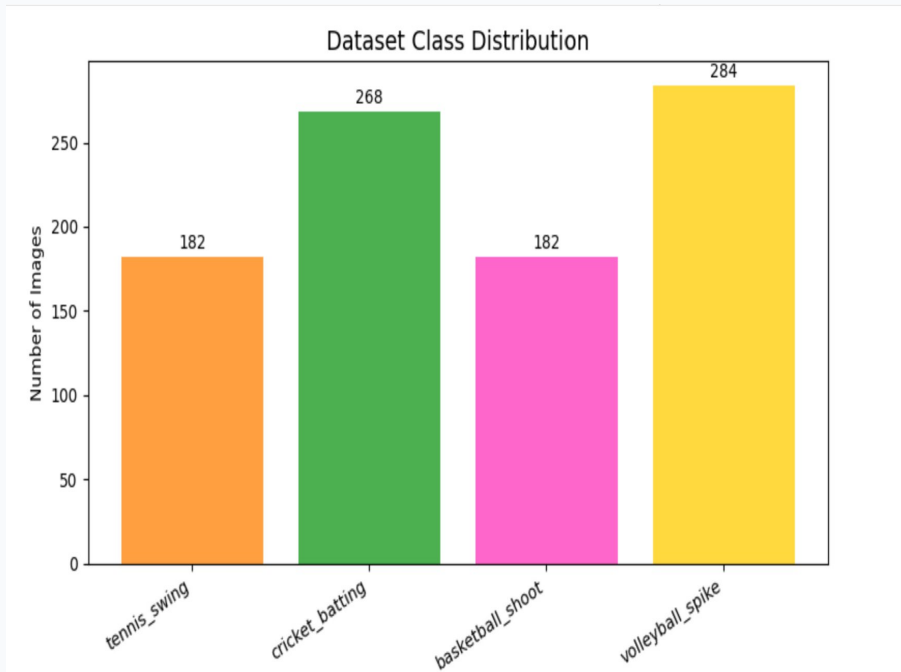
Class	Images	Training	Val	Test
Basketball Shoot	182	Train: 145	Val: 18	Test: 19
Cricket Batting	268	Train: 214	Val: 27	Test: 27
Tennis Swing	182	Train: 145	Val: 18	Test: 19
Volleyball Spike	284	Train: 227	Val: 28	Test: 29

Annotation: Bounding boxes per player • Input resolution: 640 × 640 px • Collected from publicly available sport images



Dataset Class Distribution

Visual breakdown of images per sport class



Largest Class

Volleyball Spike

284 images

Smallest Classes

Basketball & Tennis

182 images each

Label Format

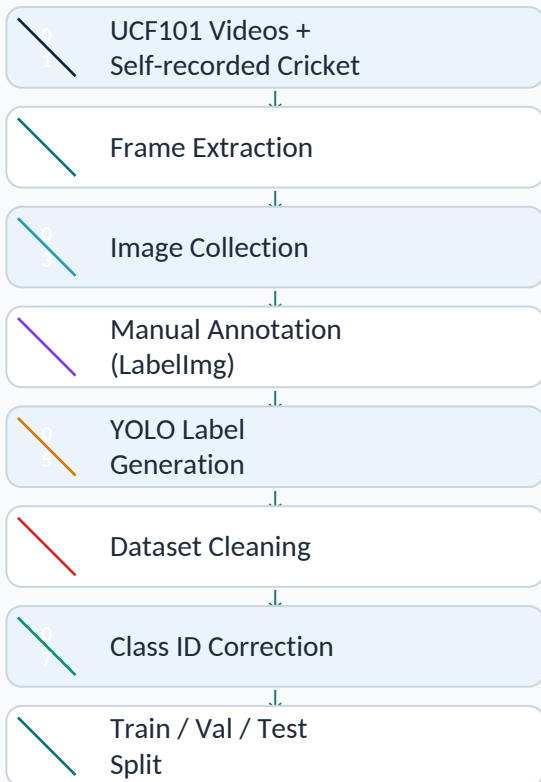
YOLO TXT

class x_c y_c w h

Critical Fix: Original labels used class IDs 0–5 (6-class scheme). Remapped to IDs 0–3 before training — this fixed mAP from 0% to 95.7%.

Dataset Collection, Cleaning & Preparation

From raw videos to a training-ready YOLO dataset



Dataset Collection

Basketball, Tennis & Volleyball: frame extraction from UCF101 action videos. Cricket Batting: UCF101 frames + self-recorded batting video. Diverse player poses across all classes.

Frame Extraction & Annotation

- Videos converted to image frames at multiple timestamps
- Redundant and blurred frames removed manually
- Images annotated with Labellmg in YOLO bounding-box format
- One .txt annotation file generated per image

Dataset Cleaning & Verification

- Removed duplicates, blurred and irrelevant images
- Deleted unused annotation files to maintain pairing
- Verified every image had a matching YOLO label file
- Corrected incorrect class IDs before training

Dataset Organization

80% Train · 10% Validation · 10% Test — Ready for YOLOv11n training

Careful dataset preparation improves annotation quality, training consistency, and model reliability.

03

Technical Pipeline

End-to-end workflow from data to live detection

Technical Pipeline

Seven-stage workflow from raw images to on-chip live detection



Training & IMX Export run in Google Colab (T4 GPU) • RPK conversion & live demo run on Raspberry Pi 5

Key constraint: Only YOLOv11n (nano) fits within the Sony IMX500 chip's on-board memory. Larger variants are rejected at RPK conversion.

04

Model Architecture

YOLOv11n — chosen for IMX500 compatibility

YOLOv11n — Model Specifications

Nano variant is the only YOLO size supported by the Sony IMX500 chip

Model Specifications

Architecture	YOLOv11n (nano)
Layers	101 (fused)
Parameters	2.58 M
GFLOPs	6.3
Input Size	640 × 640 px
Output Format	INT8 quantized
Model Size	5.2 MB
Classes	4 sport actions

Why YOLOv11n?

- IMX500 chip memory only fits nano variant
- Larger models fail at RPK conversion
- 2.58 M params — ideal for edge deployment
- Single-pass detector — no region proposals

How YOLO Works

- Divides image into grid cells
- Each cell predicts box + class in one pass
- No two-stage region proposal needed
- NMS filters overlapping detections

05

Training Setup

Google Colab T4 GPU · 100 epochs · Early stopping

Training Configuration

YOLOv11n trained on Google Colab with T4 GPU

G

Platform

Google Colab

T4 GPU · Python 3.12

E

Epochs

100 max / 79

Early stop at epoch 59

B

Batch Size

16

640×640 px input

P

Patience

20 epochs

Best model at epoch 59

O

Optimizer

Auto (SGD)

lr0=0.01 momentum=0.937

S

Image Size

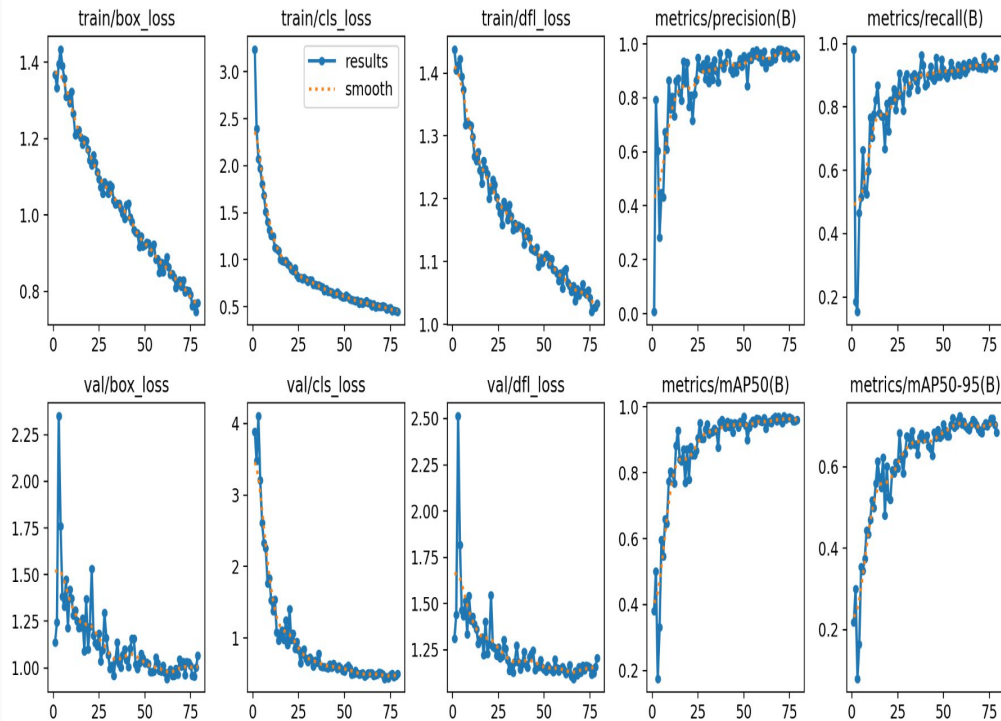
640 × 640

Standard YOLO input

Training Curves

Loss decreasing steadily · mAP50 converging to 97% · No overfitting observed

Training Results:



Loss Trend

All 3 losses (box, cls, dfl) decreasing smoothly

Best Epoch

Epoch 59 — highest mAP50
Early stop at epoch 79

No Overfitting

Train and val losses track closely throughout

06

Evaluation & Results

Metrics, confusion matrix, and per-class analysis

Model Performance — Overall Metrics

YOLOv11n • Validation set • confidence threshold 0.25

95.7%

mAP50

96.6%

Precision

90.2%

Recall

93.3%

F1 Score

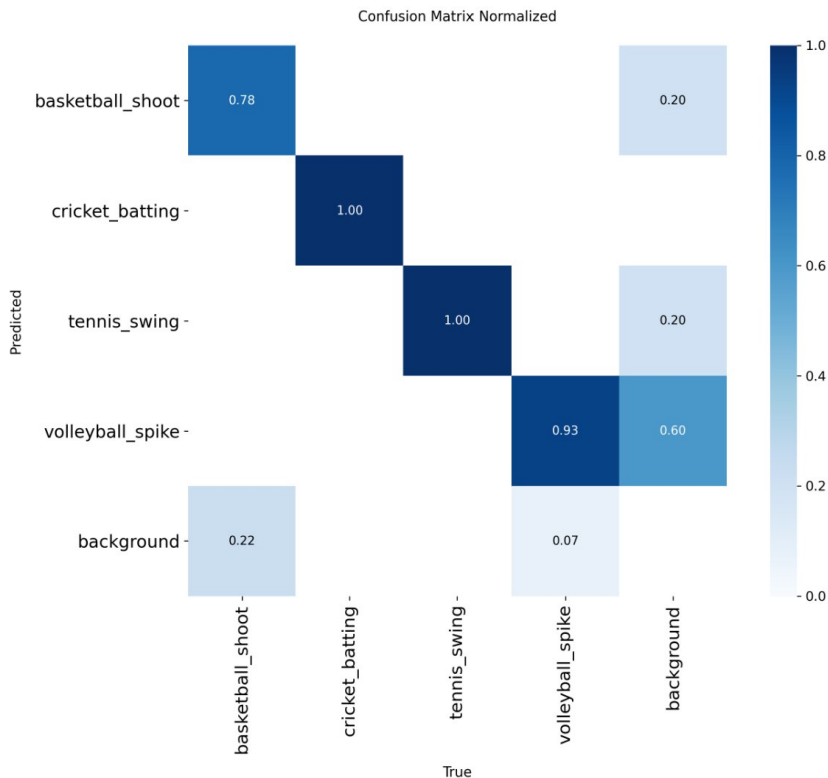
Per-Class Breakdown

Sport Class	Precision	Recall	mAP50	Status
Basketball Shoot	98.1%	75.3%	92.0%	✓ Good
Cricket Batting	98.7%	100%	99.5%	✓ Excellent
Tennis Swing	97.1%	100%	99.5%	✓ Excellent
Volleyball Spike	90.4%	85.7%	91.9%	✓ Good

Confusion Matrix

Strong diagonal — cricket batting and tennis swing at 100% recall

Confusion Matrix Normalized:



1.00

Cricket Batting
Perfect recall

1.00

Tennis Swing
Perfect recall

0.78

Basketball Shoot
20% as background

0.93

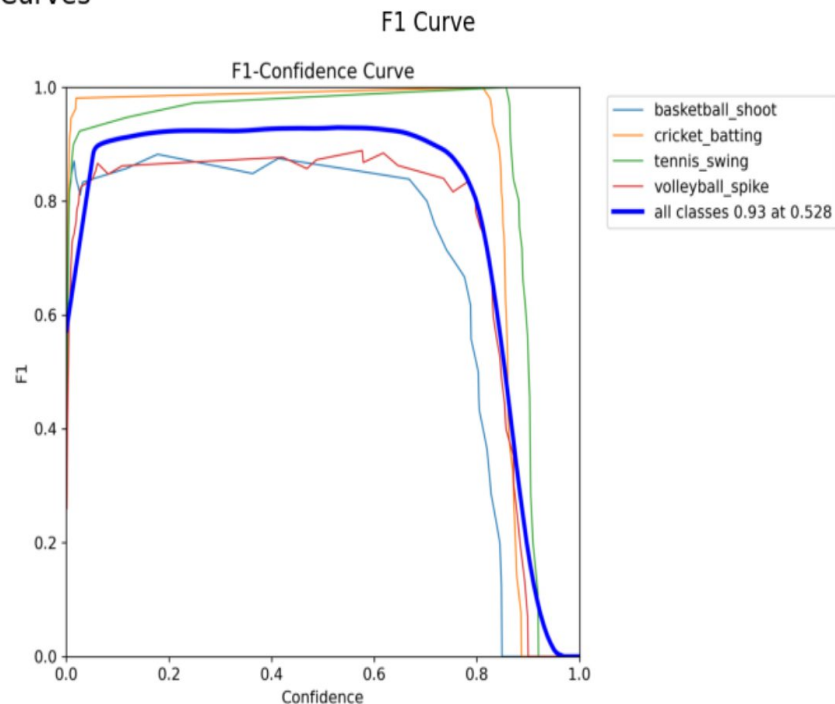
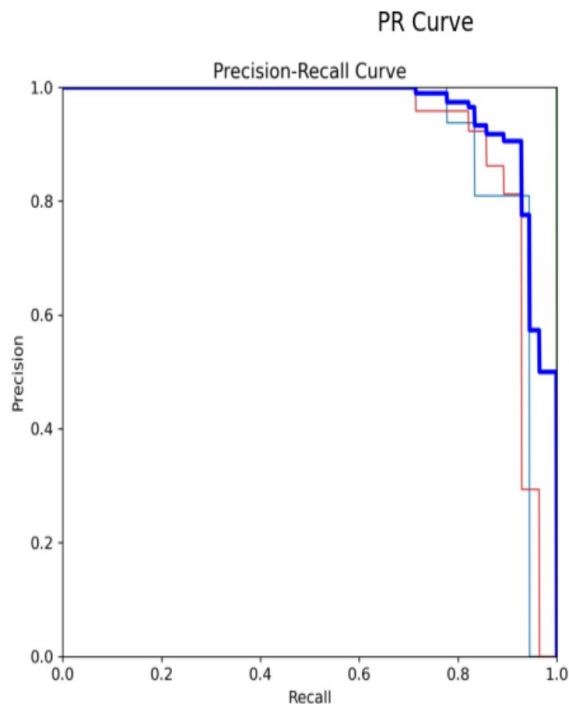
Volleyball Spike
7% missed in crowd

Precision-Recall & F1 Curves

All classes maintain high precision • F1 peaks at 0.93 @ conf 0.528

True

Training Curves



Sample Predictions — Validation Set

Model correctly identifies sport poses with tight bounding boxes



07

Deployment

From packerOut.zip to live IMX500 detection on Raspberry Pi 5

Deployment Pipeline — Raspberry Pi 5 + IMX500

Three steps on-device after training completes in Colab

Export to IMX

```
model.export(  
    format='imx',  
    int8=True)
```

**Output: packerOut.zip
+ labels.txt**

Convert to RPK

```
imx500-package  
-i packerOut.zip  
-o rpk_output/
```

Output: network.rpk

Run Live Demo

```
python3 sport_detect.py  
--model network.rpk  
--labels labels.txt  
--fps 25
```

**Output: Live detection
@ 25 FPS**

08

Constraints & Future Work

Known limitations and paths to improvement

Constraints & Limitations

Challenges encountered during the project

Dataset Size

916 images is sufficient for proof of concept. Production systems typically need 5000+ images per class.

INT8 Calibration

Export calibration used only 4 generic COCO images. Sony recommends 300+ domain-specific images for best quantization accuracy.

IMX Export Dependencies

protobuf, numpy, and edge-mdt-cl version conflicts on Python 3.12 required careful pinning. Resolved by running export in Colab.

Single-Person Detection

The model places one box per active player. Crowded scenes with multiple players visible simultaneously reduce accuracy.

Future Work & Improvements

Paths to a stronger, more capable system

01 Domain Calibration

Use 300+ sport-specific images during INT8 export calibration to close the quantization accuracy gap.

02 Larger Dataset

Collect 500+ images per class with diverse angles, lighting conditions, and backgrounds.

03 Pose Keypoints

Add skeleton keypoint estimation to better disambiguate visually similar sport poses.

04 Multi-Person Tracking

Implement ByteTrack or SORT to handle team scenarios with multiple players simultaneously.

05 FPS Benchmarking

Systematically measure on-device latency, FPS, and power consumption using IMX500 performance counters.

09

Conclusion

Project summary and key achievements

Project Summary

Complete end-to-end sport activity recognition on IMX500 edge AI camera



Dataset

916 images

4 classes · YOLO format



Model

YOLOv11n

2.58M params · 5.2MB



Training

95.7% mAP50

79 epochs · T4 GPU



Evaluation

F1: 93.3%

P: 96.6% R: 90.2%



IMX Export

INT8 Quant

packerOut.zip → RPK



Live Detection

25 FPS

On-chip · Raspberry Pi 5

YOLOv11n on Sony IMX500 achieves 95.7% mAP50 at 25 FPS — proving that compact, on-chip sport activity recognition is viable for real-time edge AI applications.

Thank You

Questions & Discussion

mAP50: 95.7% · Precision: 96.6% · Recall: 90.2% · F1: 93.3%

YOLOv11n · Sony IMX500 · Raspberry Pi 5 · 25 FPS On-Device